

Tutorial: Building the Simplest Best Digital Asset Custody Abstraction Layer

Provider-neutral custody access using a canonical API, custody core, router, provider SPI, and adapters

All provider names in this tutorial are fictional: AlphaCustody, BetaVault, and InternalSigner.

1. What problem are we solving?

Business applications need custody capabilities such as wallet creation, balance lookup, asset transfer, transaction signing, and status tracking. Without an abstraction layer, every application must integrate directly with every custody or signing provider. That creates duplicated integration logic, provider lock-in, inconsistent controls, inconsistent audit trails, and difficult migration later.

The abstraction layer solves this by giving upstream applications one stable enterprise custody API. The layer translates, controls, routes, and records the request before it reaches a downstream provider.

2. The simplest useful architecture

The simplest best architecture is intentionally small. It has five core building blocks: a canonical custody API, a canonical custody model, a custody core, a provider router, and provider adapters behind a provider-neutral SPI. This is enough to decouple applications from providers without creating an over-engineered platform.

Simplest Custody Abstraction Architecture

Canonical API + Custody Core + Router + Provider SPI + Adapters

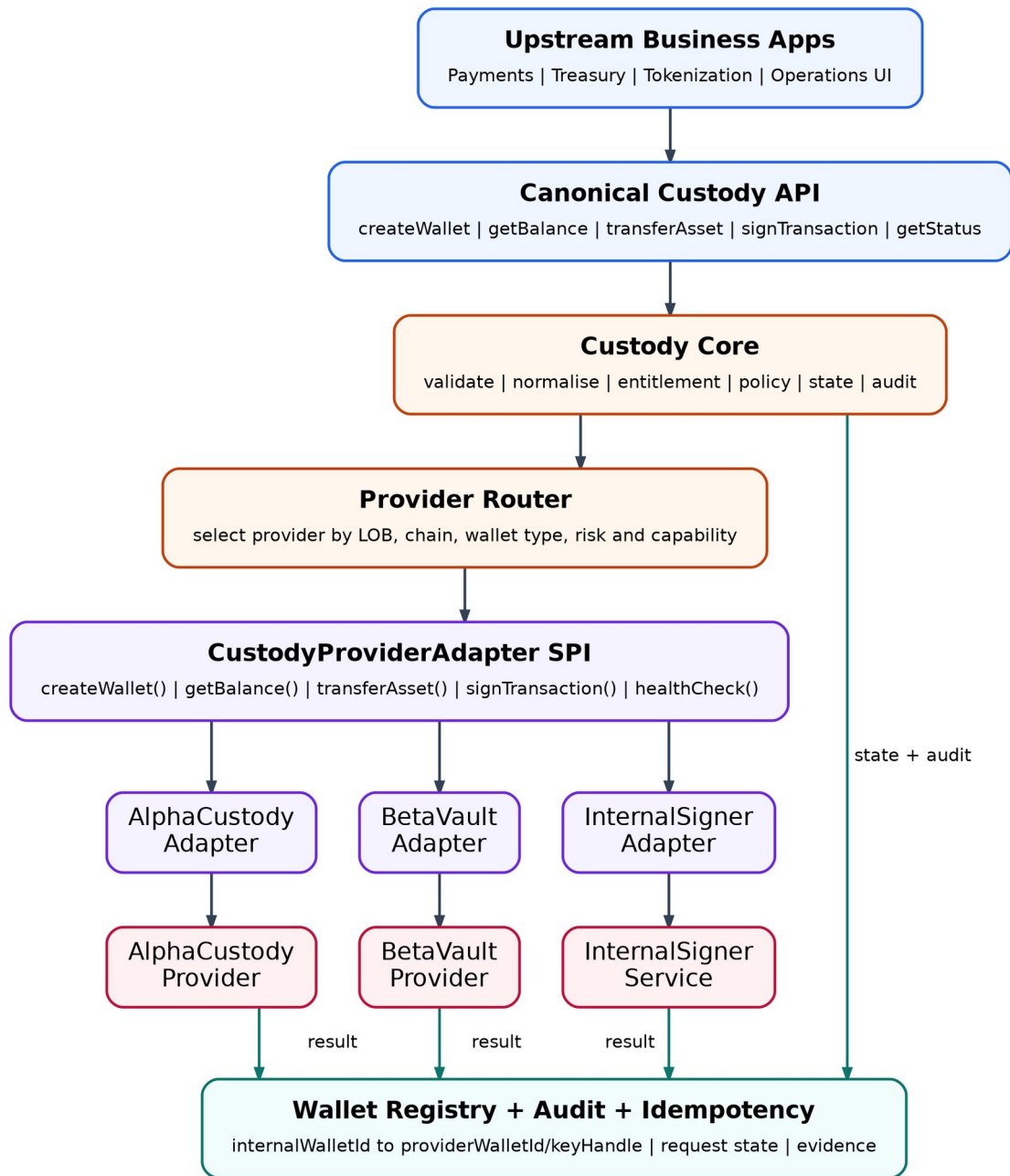


Figure 1. Simplest custody abstraction architecture using fictional providers.

3. What this architecture is called

The practical name is Digital Asset Custody Abstraction Layer. A stronger architecture-review name is Provider-Neutral Custody Control Plane. Technically, it is a custody-domain anti-corruption and orchestration layer using a canonical API, provider SPI, and provider adapters.

The mental model is simple: upstream applications express custody intent; the abstraction layer validates, normalises, authorises, applies policy, routes, translates, executes through an adapter, and returns a canonical response.

4. Core building blocks

4.1 Canonical Custody API

The canonical custody API is the northbound interface exposed to business applications. It should express custody operations, not provider-specific operations.

```
POST /custody/wallets
GET /custody/wallets/{walletId}
GET /custody/balances
POST /custody/transfers
POST /custody/transactions/sign
GET /custody/transactions/{transactionId}
```

Good API design hides provider-specific concepts. A business app should not call endpoints named after AlphaCustody, BetaVault, or InternalSigner. It should call custody operations such as createWallet, transferAsset, and signTransaction.

4.2 Canonical Custody Model

The canonical custody model is the internal common language. Each provider may use a different object model. One provider may return a walletId, another a vault reference, and another a key handle. The abstraction layer normalises these into enterprise objects.

- CustodyIntent
- WalletRef
- AssetRef
- ChainRef
- PolicyDecision
- ProviderRoute
- ExecutionRecord
- AuditRecord

Example canonical CREATE_WALLET intent:

```
{
  "intentType": "CREATE_WALLET",
  "businessUnit": "Payments",
  "clientId": "client-123",
  "chain": "ETHEREUM",
  "walletType": "HOT",
  "purpose": "CLIENT_WITHDRAWALS",
  "requestedBy": "payments-app-prod",
```

```
"idempotencyKey": "create-wallet-client-123-eth-001"
}
```

4.3 Custody Core

The custody core is the brain of the abstraction layer. It owns custody semantics and control decisions. It should not contain provider SDK logic. The core calls interfaces, not concrete provider APIs.

- Validate the request.
- Normalise it into a CustodyIntent.
- Check identity, entitlement, and wallet ownership.
- Apply basic policy rules.
- Track request state.
- Store audit evidence.

4.4 Provider Router

The provider router selects the downstream provider. Start with simple deterministic rules rather than a complex dynamic engine.

```
Payments hot wallets    to AlphaCustody
Treasury cold wallets   to BetaVault
Internal treasury signing to InternalSigner
Unsupported chain       to reject
Provider unavailable     to fail closed unless an approved fallback exists
```

4.5 Provider Adapters

Each provider gets an adapter. The adapter translates canonical requests into provider-specific API calls and converts provider responses back into canonical responses. This is the anti-corruption boundary.

- AlphaCustodyAdapter
- BetaVaultAdapter
- InternalSignerAdapter

5. The provider SPI

The provider SPI is the interface the custody core uses to call providers. Keep it small in version 1.

```
public interface CustodyProviderAdapter {
    CreateWalletResult createWallet(CreateWalletRequest request);
    BalanceResult getBalance(BalanceRequest request);
    TransferResult transferAsset(TransferRequest request);
    SigningResult signTransaction(SigningRequest request);
    TransactionStatus getTransactionStatus(StatusRequest request);
    HealthStatus healthCheck();
}
```

The core calls CustodyProviderAdapter. Only adapters know how to call AlphaCustody, BetaVault, or InternalSigner.

6. Minimum viable implementation structure

```
custody-abstraction-service/  
  
  api/  
  
  domain/  
    CustodyIntent  
    WalletRef  
    PolicyDecision  
    ProviderRoute  
    ExecutionRecord  
  
  core/  
    IntentService  
    EntitlementService  
    PolicyService  
    RoutingService  
    ExecutionStateService  
    AuditService  
  
  registry/  
    WalletRegistry  
    ProviderCapabilityRegistry  
  
  adapters/  
    AlphaCustodyAdapter  
    BetaVaultAdapter  
    InternalSignerAdapter  
  
  persistence/  
    wallet_registry_table  
    execution_state_table  
    audit_event_table  
    idempotency_table
```

Build this as one deployable service first. Do not split into many microservices until the domain boundaries and operational needs are proven.

7. Database tables required in version 1

Table	Purpose	Key fields
-------	---------	------------

wallet_registry	Maps internal wallet IDs to provider-specific references.	internal_wallet_id, provider_name, provider_wallet_id, key_handle, chain, address, status
execution_state	Tracks lifecycle of each request.	request_id, intent_id, operation, status, selected_provider, error_code
audit_event	Stores evidence for audit and investigation.	event_id, request_id, actor, policy_decision, route, payload_hash, timestamp
idempotency_record	Prevents duplicate execution on retries.	idempotency_key, request_hash, request_id, response_hash, expires_at

8. End-to-end example: createWallet

The createWallet flow shows how a business request reaches a downstream provider without exposing provider-specific semantics to the upstream application.

createWallet End-to-End Flow

Successful path plus exception outcomes

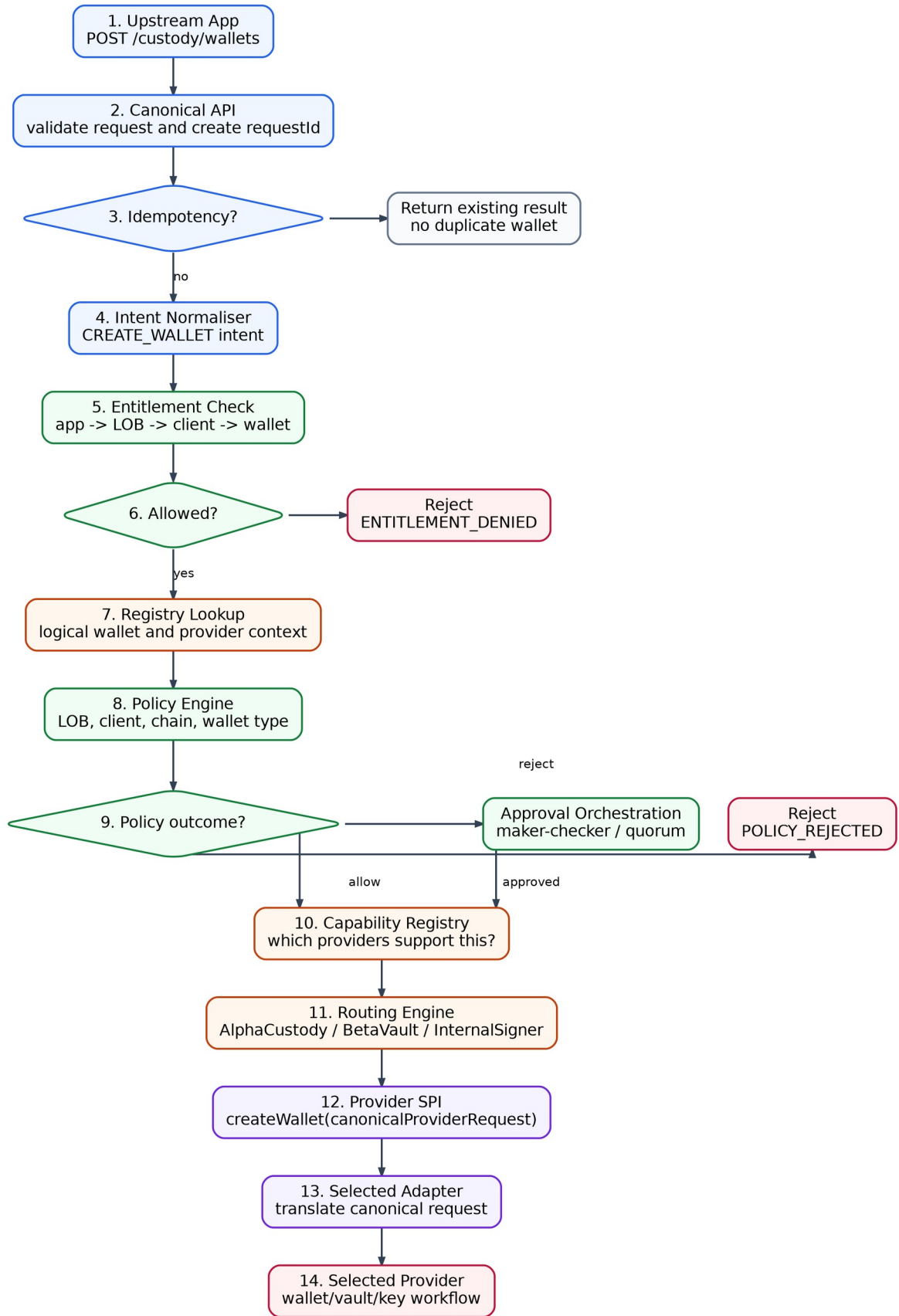


Figure 2. createWallet end-to-end flow with exception outcomes.

8.1 Request example

POST /custody/wallets

```
{
  "idempotencyKey": "create-wallet-client-123-eth-001",
  "businessUnit": "Payments",
  "clientId": "client-123",
  "chain": "ETHEREUM",
  "walletType": "HOT",
  "purpose": "CLIENT_WITHDRAWALS",
  "requestedBy": "payments-app-prod"
}
```

8.2 Step-by-step runtime path

1. Canonical API validates the request and creates an internal requestId.
2. Idempotency check ensures the same request does not create a duplicate wallet.
3. Intent Normaliser converts the request into a canonical CREATE_WALLET CustodyIntent.
4. Entitlement Check verifies the app, business unit, client, and wallet permissions.
5. Wallet Registry Lookup checks whether a logical wallet already exists and gathers provider context.
6. Policy Engine decides ALLOW, APPROVAL_REQUIRED, or REJECT.
7. Capability Registry identifies providers that support the chain, wallet type, operation, and risk tier.
8. Routing Engine selects AlphaCustody, BetaVault, or InternalSigner.
9. Custody Provider SPI calls createWallet using a canonical provider request.
10. Selected Adapter translates the canonical request into the provider-specific wallet, vault, or key workflow.
11. Provider returns a providerWalletId, vault reference, or keyHandle plus an address.
12. Abstraction layer normalises the result into internalWalletId and canonical status.
13. Wallet registry and audit evidence are updated.
14. App receives a canonical response with walletId, chain, address, and status = WALLET_CREATED.

8.3 Interaction view

The same createWallet operation can also be viewed as a simple interaction sequence between participants.

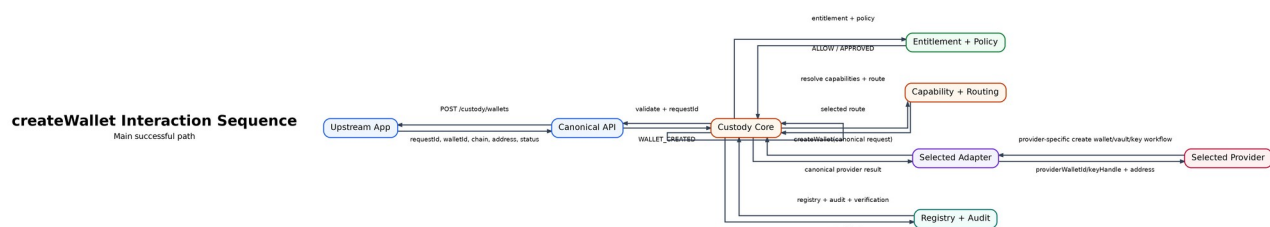


Figure 3. createWallet interaction sequence using fictional provider names.

8.4 Response example

```
{  
  "requestId": "req-001",  
  "walletId": "wallet-123",  
  "chain": "ETHEREUM",  
  "address": "0xabc123...",  
  "status": "WALLET_CREATED"  
}
```

The upstream app receives the internal walletId. Provider-specific wallet IDs, vault references, or key handles remain internal to the abstraction layer.

9. Transfer and signing operations later

After createWallet, the next operation is usually transferAsset or signTransaction. The flow is similar, but transaction operations require additional controls: transaction building/decoding, destination checks, amount checks, policy-to-payload binding, approvals for high-risk operations, and confirmation or finality tracking.

App requests transfer
to Canonical API
to Idempotency
to Normalise TRANSFER_ASSET intent
to Resolve wallet
to Build/decode transaction
to Policy and approval checks
to Route provider
to Adapter executes transfer/signing
to Provider returns signature or transaction hash
to Audit, state update, and canonical response

10. What to include in version 1 and what to defer

Capability	Version 1?	Reason
Canonical API	Yes	Main decoupling point for upstream applications.
Canonical model	Yes	Creates one enterprise custody language.
Entitlement check	Yes	Prevents unauthorised wallet and signing actions.
Basic policy	Yes	Controls wallet type, chain, client, business unit, and risk tier.
Wallet registry	Yes	Maps internal IDs to provider-

		specific references.
Provider router	Yes	Selects the provider using simple rules.
Provider adapters	Yes	Isolates provider APIs.
Execution state	Yes	Tracks lifecycle and errors.
Audit log	Yes	Provides evidence.
Idempotency	Yes	Prevents duplicate execution.
Webhook ingestion	Later	Add when asynchronous provider events are needed.
Advanced approvals	Later	Start simple unless the use case requires complex workflows.
On-chain verification	Later for wallet creation; earlier for transfers	Wallet creation may only require provider state/address verification.
Governance UI	Later	Add when routing and policy changes need operational administration.

11. What to avoid

- Do not build a large platform before the first use case is proven.
- Do not expose provider-specific IDs, statuses, errors, or APIs to upstream apps.
- Do not put provider SDK calls inside the custody core.
- Do not start with a complex policy language if simple rules are sufficient.
- Do not split the design into many microservices before the domain is stable.

12. Final design principle

The single most important rule is: upstream applications must depend only on canonical custody contracts. Provider-specific APIs, IDs, statuses, errors, and workflow semantics remain behind adapters.

A good version 1 consists of a canonical API, canonical model, entitlement, basic policy, routing rules, provider SPI, two provider adapters, wallet registry, idempotency, execution state, and audit evidence. This creates strategic decoupling without turning the first release into a monster platform.

13. Final architecture statement

The custody abstraction layer exposes a canonical custody API to upstream applications and hides the implementation details of downstream custody and signing providers. Business requests are normalised into canonical custody intents, checked against entitlement and policy, routed to the appropriate provider through a provider-neutral adapter interface, translated by provider-specific adapters, and returned as canonical responses. The system maintains internal wallet-provider mappings, execution state, idempotency records, and audit evidence so that upstream applications remain decoupled from provider-specific APIs and semantics.